

T1: Basic Searching and Sorting in C

We'll learn about two basic searching algorithms—Linear Search and Binary Search—as well as two sorting algorithms—Selection Sort and Counting Sort. You'll be provided with prototype code where you'll fill in the blanks and add comments to understand how these algorithms work.

1. Linear Search

Linear Search is the simplest searching algorithm. It works by checking each element in the list one by one until the desired element is found or the list ends.

```
#include <stdio.h>

int linearSearch(int arr[], int n, int target) {
    // Loop through each element in the array
    for (int i = 0; i < n; i++) {
        // If the current element matches the target, return its index
        if (/* condition to check if arr[i] is equal to target */) {
            return i;
        }
    }
    // If the target is not found, return -1
    return -1;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 30;

    int result = linearSearch(arr, n, target);
    if (result != -1) {
        printf("Element found at index %d\n", result);
    } else {
        printf("Element not found\n");
    }

    return 0;
}
```

2. Binary Search

Binary Search is more efficient but requires the list to be sorted. It works by repeatedly dividing the list in half until the target element is found.

```
#include <stdio.h>

int binarySearch(int arr[], int low, int high, int target) {
    while (low <= high) {
        int mid = /* calculate the middle index */;

        // Check if the middle element is the target
        if (/* condition to check if arr[mid] is equal to target */) {
            return mid;
        }
        // If target is greater, ignore the left half
        if (/* condition to check if arr[mid] is less than target */) {
            low = mid + 1;
        }
        // If target is smaller, ignore the right half
        else {
            high = mid - 1;
        }
    }
    return -1;
}

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 30;

    int result = binarySearch(arr, 0, n - 1, target);
    if (result != -1) {
        printf("Element found at index %d\n", result);
    } else {
        printf("Element not found\n");
    }

    return 0;
}
```

3. Selection Sort

Selection Sort repeatedly selects the smallest element from the unsorted part of the list and moves it to the front.

```
#include <stdio.h>

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        int min_idx = i;
        for (int j = i+1; j < n; j++) {
            if (/* condition to check if arr[j] is smaller than
arr[min_idx] */) {
                min_idx = j;
            }
        }
        // Swap the found minimum element with the first element
        int temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }
}

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    selectionSort(arr, n);

    printf("Sorted array: \n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

4. Counting Sort

Counting Sort is a non-comparison-based sorting algorithm that works well when the range of input elements is known and not too large.

```
#include <stdio.h>
void countingSort(int arr[], int n) {

    int output[n];
    int count[10] = {0}; // Assuming the elements are in the range 0-9
    // the count of each integer value is initialized as 0

    // Count the occurrence of each element
    for (int i = 0; i < n; i++) {
        count[arr[i]]++;
    }

    // Print the counts of each element
    for (int i = 0; i < 10; i++) //Element
    {
        for(int j = 0; j< count[i]; j++) //Count of the element
        {
            printf("%d", i);
        }
    }
}

int main() {
    int arr[] = {4, 2, 2, 8, 3, 3, 1};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Sorted array: \n");
    countingSort(arr, n);

    return 0;
}
```

Sample Problems for Next Week's Foundation

Problem 1: Inserting an Element in a Sorted Array

- **Task:** Write a function that inserts an element into its correct position in a sorted array.
- **Example:** Given a sorted array `arr = {1, 3, 5, 7}`, insert the element `4` to maintain the sorted order.
- **Sample Output:** `arr = {1, 3, 4, 5, 7}`

Problem 2: Merging Two Sorted Arrays

- **Task:** Combine two sorted arrays into one sorted array.
- **Example:** Given `arr1 = {1, 3, 5}` and `arr2 = {2, 4, 6}`, merge them.
- **Sample Output:** `mergedArray = {1, 2, 3, 4, 5, 6}`

```
#include <stdio.h>

int main() {

    int arr1[] = {10, 20, 29, 35, 42};
    int n1 = sizeof(arr1) / sizeof(arr1[0]);

    int arr2[] = {3, 8, 11};
    int n2 = sizeof(arr2) / sizeof(arr2[0]);

    int p_arr1 = 0;
    int q_arr2 = 0;

    while(p_arr1 < n1 && q_arr2 < n2)
    {
        if (arr1[p_arr1] > arr2[q_arr2])
        {
            printf("%d ", arr2[q_arr2]);
            q_arr2++;
        }
        else
        {
            printf("%d ", arr1[p_arr1]);
            p_arr1++;
        }
    }
}
```

```

while (p_arr1 < n1)
{
    printf("%d ", arr1[p_arr1]);
    p_arr1++;
}

while(q_arr2 < n2)
{
    printf("%d ", arr2[q_arr2]);
    q_arr2++;
}

return 0;
}

```

Problem 3: Store the Minimum/Maximum at the last location by Comparing Adjacent Elements

- **Task:** Repeatedly compare adjacent elements and put the maximum value in the last location.
- **Example:** Given an array `arr = {6, 3, 5, 1}`.
 - Start from the beginning and iterate over the elements one by one.
 - Swap adjacent elements if the element on the right is smaller than the element on the left.
- **Output:** `arr = {3, 5, 1, 6}`

Problem 4: Partitioning the Array

- **Task:** Rearrange an array so that all elements less than a given pivot are on its left side, and all elements greater are on its right.
 - **Example:** For `arr = {7, 2, 1, 6, 8, 5, 3, 4}` with pivot `6`, rearrange the array.
 - **Sample Output:** `arr = {2, 1, 5, 3, 4, 6, 8, 7}`
-